Tarea 2: Árboles B y B^ε en memoria externa

CC4102 Diseño y Análisis de Algoritmos · Semestre 2026-2

Julio Yáñez · 15 de octubre de 2026

1. Introducción

En memoria externa, el costo de una operación se mide en transferencias de bloques entre el disco y la memoria principal (I/Os). El árbol B es la estructura clásica para un diccionario ordenado en este modelo: cada nodo ocupa un bloque con $\Theta(B)$ llaves, por lo que insertar y buscar cuesta $\Theta(\log_B N)$ I/Os. El árbol B^ε cambia ese equilibrio a favor de las inserciones: cada nodo interno usa solo del orden de B^ε llaves divisorias y destina el resto del bloque a un *buffer* de inserciones pendientes, que bajan a los hijos en lotes cuando el buffer se llena. Así, cada I/O de un vaciado mueve muchos pares a la vez y una inserción cuesta $O(\log_B N/(\varepsilon B^{1-\varepsilon}))$ I/Os amortizadas, a cambio de una altura $O(\log_B N/\varepsilon)$, que es lo que pagan las búsquedas.

En esta tarea se implementaron en C++ un árbol B ($b = 340$) y dos árboles B^ε, todos con nodos de 4096 bytes: uno con $\varepsilon \approx 0{,}25$ ($c = 4$, $l_{\max} = 506$) y otro con $\varepsilon \approx 0{,}75$ ($c = 126$, $l_{\max} = 384$). Con ellos se indexaron los primeros $N \in \{2^{15}, ..., 2^{26}\}$ pares (tiempo Unix, temperatura) medidos en la estación Quinta Normal entre 2019 y 2025. Los árboles se construyen sobre un arreglo de bloques en RAM que simula el disco y cuenta cada lectura y escritura; después se guardan en un archivo binario y se hacen 50 búsquedas de rangos de una semana leyendo los nodos desde ese archivo. Se reportan el tiempo y las I/Os de la construcción, el tamaño y la altura de cada árbol, y el tiempo y las I/Os de las búsquedas. En el experimento bonus se estudia además el parámetro α del vaciado con $N = 2^{24}$.

La Sección 2 describe las estructuras, los algoritmos y las decisiones de implementación que el enunciado deja abiertas. La Sección 3 presenta el entorno, la metodología y los resultados; la Sección 4 los analiza con el análisis amortizado de los vaciados y la altura en función de ε, y la Sección 5 concluye.

Hipótesis. Un bloque de 4096 bytes tiene espacio para $B = 512$ pares de 8 bytes (el nodo del árbol B guarda 340, porque también almacena los índices de los hijos), y $B^{0{,}25} \approx 4{,}8$ y $B^{0{,}75} \approx 108$, que corresponden aproximadamente a los 5 y 127 hijos de las dos configuraciones.

1. *Inserción.* El árbol B lee el camino desde la raíz y escribe la hoja: unas $h + 1$ I/Os por inserción, con una altura h de 2 a 4 para estos N. En un árbol B^ε cada inserción lee y escribe la raíz (2 I/Os) y los vaciados agregan poco, porque cada lote mueve muchos pares: con $\varepsilon \approx 0{,}25$ el hijo más poblado recibe al menos $\lceil 506/5 \rceil = 102$ pares por vaciado, pero con $\varepsilon \approx 0{,}75$ solo $\lceil 384/127 \rceil = 4$. Se espera entonces, en I/Os por inserción, B^ε (0,25) $<$ B^ε (0,75) $<$ B, con el primero muy cerca de 2. Como en la simulación cada I/O copia un bloque de 4 KiB, el tiempo de creación debería seguir el mismo orden.
2. *Altura y búsqueda.* La altura de un B^ε crece como $1/\varepsilon$: el árbol con $\varepsilon \approx 0{,}25$ debería tener unas 3 veces la altura del de 0,75, que a su vez debería ser casi tan bajo como el árbol B. En búsquedas que tocan pocos nodos eso ordena los árboles como B $\approx$ B^ε (0,75) $<$ B^ε (0,25). Pero un rango de una semana contiene $\approx 0{,}29$ % de los datos ($\approx 195\,000$ pares con $N = 2^{26}$, cientos de bloques), así que para N grande debería dominar el número de bloques con resultados y los tres árboles deberían acercarse.
3. *Tamaño.* Las hojas de los B^ε tienen espacio para más pares que las del árbol B (506 y 384 contra 340), así que deberían ocupar menos bloques, en especial con $c = 4$.

2. Desarrollo

2.1. Organización del código

Los árboles y los experimentos están en C++17 (`src/`, unas 2 200 líneas). El código de los gráficos está aparte, en Python (`graficos/`), y solo lee los CSV que produce el programa. Los árboles son plantillas parametrizadas por b o por $(c, l_{\max})$: el programa usa $b = 340$, $(4, 506)$ y $(126, 384)$, y las pruebas usan valores chicos como $b = 4$ o $(c, l_{\max}) = (4, 6)$, que generan árboles profundos con pocos datos y ejercitan

todos los casos de split y vaciado. Cada estructura y función está documentada en el código con su propósito y sus parámetros de entrada y salida.

Archivo	Contenido
`par.hpp`, `arbol_comun.hpp`	par llave-valor de 8 bytes (mismo formato que `datos.bin`), inserción ordenada y mezcla de lotes
`disco.hpp`	disco simulado en RAM: lecturas y escrituras de bloques con conteo de I/Os; escritura del árbol a archivo
`archivo.hpp`	lectura de un nodo desde el archivo del árbol (`pread` con `O_DIRECT`)
`arbol_b.hpp`, `arbol_be.hpp`	árbol B y árbol B^ε: splits, vaciado, inserción y búsqueda por rango
`datos.hpp`, `main.cpp`	lectura de `datos.bin`; experimentos, consultas puntuales y escritura de los CSV
`tests/pruebas.cpp`	pruebas de correctitud (invariantes, I/Os exactas, comparación con fuerza bruta)

Tabla 1: Archivos del programa en C++.

2.2. Nodos, disco simulado y archivo

Los nodos son `struct` con exactamente los campos del enunciado y en el mismo orden (`NodoB<340>`, `NodoBe<4,506>` y `NodoBe<126,384>`); un `static_assert` comprueba que cada uno mide 4096 bytes. Como no contienen punteros, un nodo se guarda y se lee copiando sus bytes.

El disco simulado es un arreglo de nodos en RAM con tres operaciones, y los algoritmos solo acceden a los nodos a través de ellas: `leer(i, nodo)` copia el bloque i completo a una variable local y cuenta una lectura; `escribir(i, nodo)` copia la variable al bloque i y cuenta una escritura; y `agregar(nodo)` la escribe al final, cuenta una escritura y retorna su posición. Como los algoritmos trabajan siempre sobre copias locales, igual que si hubieran traído el bloque del disco, los contadores son exactamente la cantidad de bloques transferidos. La raíz está siempre en el bloque 0.

El arreglo no es un `std::vector`. Al crecer, un vector reserva un arreglo del doble de tamaño y copia el anterior, y con el árbol B de $N = 2^{26}$ (1,1 GiB) eso habría necesitado por un momento unos 2 GiB (el arreglo viejo de 1 GiB más su copia), más de lo que tenía libre la máquina (Sección 3.1). En su lugar, los bloques se guardan en segmentos de 1024 nodos (4 MiB) reservados con `mmap`, que se agregan cuando hacen falta y se devuelven al sistema operativo al destruir el árbol. Para los algoritmos sigue siendo un arreglo indexado por posición.

Al terminar las inserciones, el árbol se escribe en un archivo binario, bloque por bloque y en orden (el nodo i queda en el byte $i \cdot 4096$), se hace `fsync` y se le pide al sistema operativo que saque el archivo de su caché de páginas (`posix_fadvise`). Después se destruye el árbol en RAM. Las búsquedas abren el archivo con `O_DIRECT`: leer el nodo i es un `pread` de 4096 bytes en la posición $i \cdot 4096$ hacia un buffer alineado, que va al dispositivo sin pasar por la caché del sistema operativo. Sin `O_DIRECT`, después de la primera búsqueda el archivo quedaría en esa caché y se mediría la velocidad de la RAM. Cada nodo leído es una I/O de búsqueda.

2.3. Árbol B

La inserción sigue el esquema del enunciado, con división preventiva: antes de bajar a un hijo lleno ($k = b$), se lo divide en el par de la posición $b/2 = 170$, que deja 169 pares a la izquierda y 170 a la derecha. U_{der} se agrega al final, U_{izq} reemplaza a U y el padre, que siempre tiene espacio porque nunca se entra a un nodo lleno, recibe la mediana y se escribe. La inserción continúa en la mitad que corresponde, que ya está en memoria y no se vuelve a leer. Una raíz llena se divide igual: ambas mitades van al final del arreglo y la nueva raíz al bloque 0.

La búsqueda por rango recorre cada nodo en orden (hijo, par, hijo, …) y baja solo a los hijos cuyo rango intersecta $[l, u]$. Con a el primer índice cuya llave es $\geq l$ y z el primero cuya llave es $> u$ (búsquedas binarias), visita los hijos $a, ..., z$ y reporta los pares $a, ..., z - 1$. Por eso el resultado queda ordenado por llave.

2.4. Árbol B^ε

Inserción. Cada inserción lee la raíz y la escribe con el par agregado a su buffer. Si la raíz es una hoja llena, se divide en la posición $l_{\max}/2$, como indica el enunciado, y se crea una raíz interna con el par en su buffer. Si la raíz es interna y su buffer está lleno, primero se vacía; si con eso llega a c llaves, se divide y se crea una raíz nueva en el bloque 0.

Vaciado de un nodo V. Como el buffer está ordenado, el grupo de cada hijo es un tramo contiguo, cuyos límites se encuentran con una búsqueda binaria por llave divisoria. Con $f = \lceil k_{\text{buffer}}/(k_{\text{llaves}}+1)\rceil$ se eligen los hijos que tienen al menos $\lceil \alpha f \rceil$ pares; los pares de los demás se quedan en V. Los hijos elegidos se procesan de derecha a izquierda: para cada uno se lee el hijo U y se le entrega su lote de m pares.

- Si U es interno y $k_{\text{buffer}}(U) + m > l_{\max}$, se vacía U (recursivamente) hasta que el lote quepa. Después se mezcla el lote con el buffer de U (mezcla de dos arreglos ordenados).
- Si U es una hoja y el lote no cabe, se divide la hoja (ver abajo).

Durante su vaciado, V se mantiene en memoria como una *copia de trabajo* y se escribe una sola vez, al final, por quien lo llamó. Así, el vaciado no agrega I/Os propias (el nodo ya está en memoria) y cada lote cuesta 1 lectura y 1 escritura del hijo, más 1 escritura por cada nodo nuevo que aparezca al dividirlo (Tabla 2).

Decisiones propias en los splits.

- *Split de hoja al recibir un lote.* El enunciado divide la hoja en la posición $l_{\max}/2$ y después reparte el lote, pero si el lote cae entero a un lado esa mitad puede quedar con más de $l_{\max}$ pares. En vez de eso, se mezclan los $s = k_{\text{buffer}} + m$ pares ($s \le 2l_{\max}$) y se cortan en la mediana: los primeros $\lceil s/2 \rceil$ quedan en U_{izq}, en la posición de U, y el resto en U_{der}, al final del arreglo. La llave del último par de U_{izq} sube como divisoria. Es lo mismo que dividir y repartir, pero garantiza que ambas hojas quedan con entre $l_{\max}/2$ y $l_{\max}$ pares.
- *Split de nodo interno.* Si en un mismo vaciado se dividen varios hijos, V recibe varias llaves nuevas. Como V está en memoria, se aceptan todas y, antes de escribirlo, se divide en su llave mediana (la de índice $\lfloor (K-1)/2 \rfloor$ de sus K llaves, desde 0), repitiendo en cada mitad que siga con más de c llaves; el buffer se reparte en $\le k_m$ y $> k_m$. Con $K = c+1$ quedan $c/2$ llaves en cada mitad (2 y 2 con $c = 4$; 63 y 63 con $c = 126$). Con $K = c$, el caso habitual de la raíz (que se divide en cuanto tiene c o más llaves), resultan las $c/2 - 1$ y $c/2$ llaves del enunciado. Las mitades nuevas se agregan al final y sus llaves se insertan en el padre justo a la derecha del hijo dividido; como los hijos se procesan de derecha a izquierda, los índices de los que faltan no cambian.
- *Umbral exacto.* α se guarda como el entero $\alpha \cdot 10^6$ y $\lceil \alpha f \rceil$ se calcula con aritmética entera, porque en punto flotante, por ejemplo, $100 \cdot 0{,}07 = 7{,}000000000000001$ y el techo daría 8 en vez de 7.

Búsqueda. En cada nodo se reportan los pares del buffer que están en $[l, u]$ (dos búsquedas binarias) y, si es interno, se baja a los hijos cuyo rango intersecta $[l, u]$, como en el árbol B. El resultado no queda ordenado, porque el buffer de un nodo puede tener pares posteriores a los de sus hijos; se ordena solo al escribirlo en un CSV.

Llaves repetidas. Los datos tienen 11 llaves que aparecen dos veces. Un split puede dejar pares con la misma llave a ambos lados de una llave divisoria, así que en ambos árboles se visita un hijo si su rango *cerrado* $[k_{i-1}, k_i]$ intersecta $[l, u]$. Las pruebas incluyen datos con muchas llaves repetidas y con una sola llave.

Operación	Lecturas	Escrituras
Árbol B: insertar un par sin splits (altura h)	h	1
Árbol B: split de un nodo o de la raíz	0	3
B^ε: insertar un par en el buffer de la raíz	1	1
B^ε: entregar un lote a un hijo	1	1
B^ε: cada nodo nuevo que crea un split	0	1
Búsqueda (ambos): cada nodo visitado, desde el archivo	1	0

Tabla 2: I/Os que cuenta la implementación por cada operación.

2.5. Experimentos y verificación

El ejecutable `build/tarea2` tiene los comandos `principal` ($N = 2^{15}, ..., 2^{26}$ con los tres árboles), `alpha` (bonus) y `consulta`/`buscar` (una búsqueda puntual). Para cada N y cada árbol:

1. se leen los primeros N pares de `datos.bin` en bloques de 65 536 y se insertan uno a uno; el tiempo de creación (`steady_clock`) cuenta solo las inserciones, no la lectura del archivo;
2. se registran las I/Os simuladas y la estructura del árbol (nodos, altura, ocupación, vaciados, lotes y splits);

3. se guarda el árbol en disco y se libera su RAM;
4. se hacen las búsquedas sobre el archivo, midiendo el tiempo y las I/Os de cada una.

Los 50 rangos se generan una vez con `mt19937_64` y semilla fija: l uniforme en $[1\,546\,300\,800, 1\,754\,006\,400]$ y $u = l + 604\,800$. Son los mismos para todos los N y árboles, lo que permite comparar una misma búsqueda.

Cada búsqueda se compara con una búsqueda por fuerza bruta sobre los primeros N pares (cantidad de pares, suma de llaves y suma de los bits de las temperaturas): las 5 400 búsquedas del experimento principal y las 960 del bonus coincidieron. Además, `tests/pruebas.cpp` hace 30 712 verificaciones: cuenta las I/Os exactas de casos pequeños hechos a mano; valida todas las invariantes después de cada inserción en árboles chicos (orden, rangos de los subárboles, ocupación, hojas al mismo nivel, altura, ningún nodo perdido y el mismo multiconjunto de pares que se insertó), y compara búsquedas con fuerza bruta sobre datos aleatorios, ordenados, con llaves repetidas y extremas, y sobre una muestra de `datos.bin`. Las pruebas también pasan con AddressSanitizer y UBSan.

3. Resultados

3.1. Entorno

Componente	Valor
Sistema operativo	Debian GNU/Linux forky/sid, kernel Linux 7.1.8 (x86-64)
Máquina	máquina virtual KVM con 1 vCPU Intel Core i3-6100 a 3,70 GHz
Cachés (según la VM)	L1d 32 KiB, L1i 32 KiB, L2 4 MiB, L3 16 MiB, línea de 64 B[1]
RAM	3,8 GiB (entre 1,0 y 1,9 GiB disponibles durante los experimentos); swap de 1,7 GiB
Disco	disco virtual (QEMU) de 32 GB con ext4; lectura de 4 KiB con `O_DIRECT`: mediana 0,24 ms (aleatoria) y 0,26 ms (secuencial), en 3 000 lecturas
Compilador	g++ 15.3.0, `-std=c++17 -O2`

Tabla 3: Entorno de los experimentos.

3.2. Datos y metodología

`datos.bin` (512 MiB, con el sha256 publicado) contiene $2^{26} = 67\,108\,864$ pares en orden aleatorio, con llaves entre 1 546 300 802 (1-1-2019 00:00:02 UTC) y 1 754 006 394 (31-7-2025 23:59:54 UTC), 11 llaves repetidas y temperaturas entre −2,53 y 38,32 °C. Para cada N se usan los primeros N pares; como el archivo está desordenado, cada prefijo es una muestra aleatoria de todo el período y está contenido en los prefijos más grandes.

Experimento principal. Cada combinación (N, árbol) se construyó 3 veces (108 construcciones) y cada construcción se siguió de las mismas 50 búsquedas. Las I/Os, nodos y alturas fueron idénticos en las 3 repeticiones, porque el proceso es determinista. El tiempo de creación que se reporta es la *mediana* de las 3 repeticiones, y las figuras muestran además el mínimo y el máximo. El tiempo y las I/Os de búsqueda son el promedio de las 150 búsquedas. Los árboles B^ε usan $\alpha = 0{,}9$, elegido en el bonus (Sección 3.7).

Bonus. Con $N = 2^{24}$, para cada árbol B^ε y cada uno de 8 valores de α (0,1; 0,25; 0,4; 0,5; 0,6; 0,75; 0,9 y 0,99) se hicieron 3 construcciones, cada una seguida de las mismas 20 búsquedas (generadas con otra semilla).

Ruido. La máquina era compartida: durante las corridas había otros procesos de usuario activos (por ejemplo, una batería de pruebas de otro proyecto) y la memoria estaba casi llena (con $N = 2^{26}$ la swap llegó al 100 %). Como una carga externa solo puede alargar una medición, se usa la mediana. Por ejemplo, con $N = 2^{26}$ la primera repetición del árbol B tardó 162,9 s y la del B^ε con $c = 4$ 41,8 s, contra 77,8–83,2 s y 25,5–26,1 s de las otras dos.

[1] Son los valores que expone el hipervisor. El i3-6100 físico tiene L1d y L1i de 32 KiB y L2 de 256 KiB por núcleo, y L3 de 3 MiB compartida.

3.3. Creación

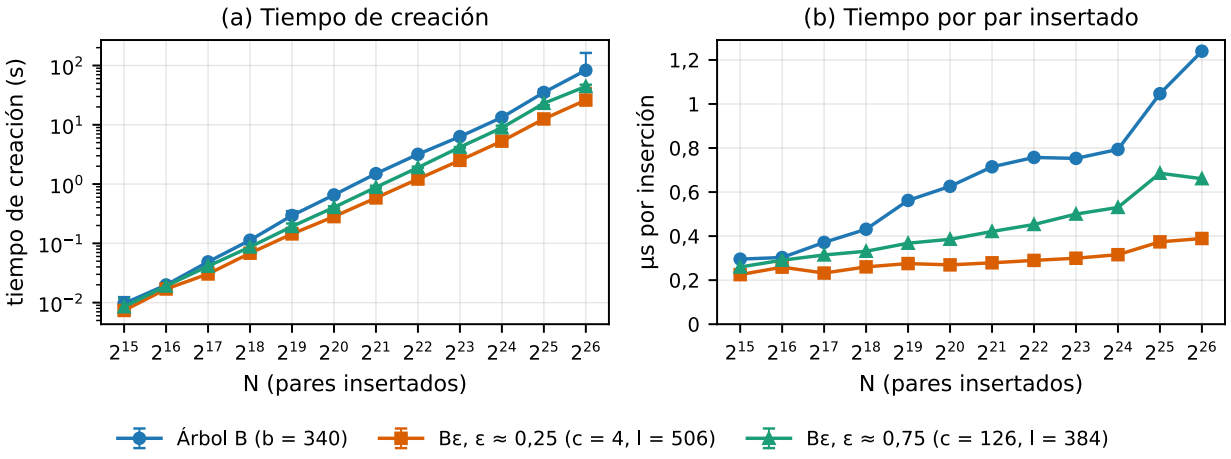


Figura 1: Tiempo de creación (inserción de los N pares) en función de N: (a) total, mediana de 3 repeticiones con barras de mínimo a máximo; (b) por par insertado.

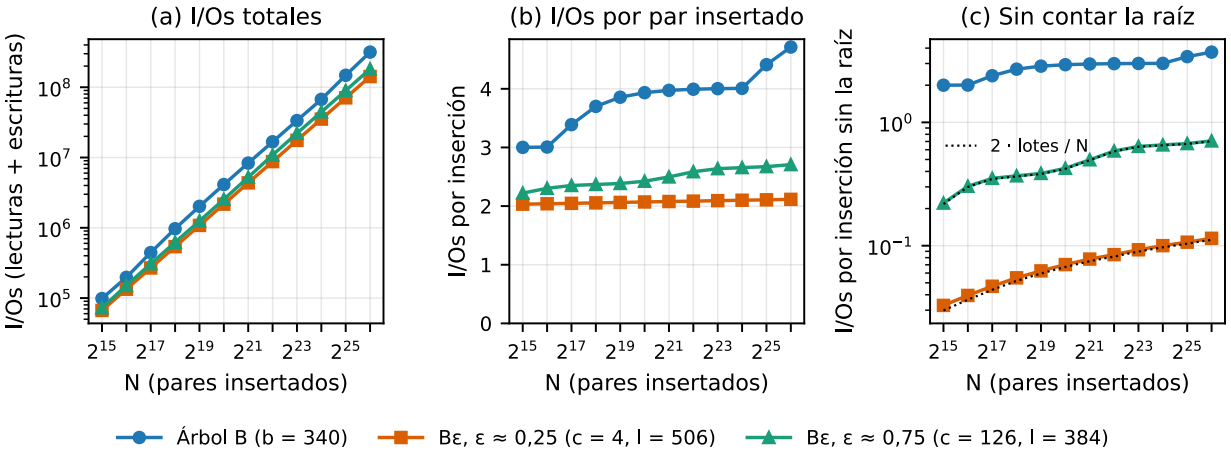


Figura 2: I/Os simuladas al insertar los N pares: (a) totales; (b) por par insertado; (c) por par sin contar la raíz (1 lectura en el árbol B, 1 lectura y 1 escritura en los B^ε). La línea punteada es $2 \cdot \text{lotes}\ /N$: leer y escribir el hijo en cada lote de un vaciado.

- El tiempo de creación crece casi linealmente con N (Figura 1 a) y, para todo $N \geq 2^{16}$, el orden es B^ε $c = 4 <$ B^ε $c = 126 <$ B. Con $N = 2^{26}$: 26,1 s, 44,3 s y 83,2 s.
- Por par insertado (Figura 1 b), el árbol B pasa de 0,30 a 1,24 µs, con un salto en 2^{25}; el B^ε $c = 126$ de 0,26 a 0,66 µs, y el B^ε $c = 4$ de 0,23 a 0,39 µs.
- Las I/Os por inserción (Figura 2 b) del árbol B suben en escalones: 3,00 con $N \leq 2^{16}$; entre 3,4 y 4,0 hasta 2^{24}, y 4,41 y 4,71 en 2^{25} y 2^{26}. Las del B^ε $c = 126$ suben de 2,22 a 2,71, y las del B^ε $c = 4$ de 2,03 a 2,12. El orden es el mismo que el del tiempo para todo N.
- Sin contar la raíz (Figura 2 c), con $N = 2^{26}$ el árbol B hace 3,71 I/Os por inserción, el B^ε $c = 126$ 0,71 y el B^ε $c = 4$ 0,115, es decir, 5,3 y 32 veces menos que el árbol B. En ambos B^ε, la línea $2 \cdot \text{lotes}\ /N$ queda sobre la curva medida.

Medida	B	B^ε c=4	B^ε c=126
Nodos internos	1 041	64 893	3 094
Hojas	278 430	139 200	250 234
% de los pares en nodos internos	0,41	29,87	1,14
Pares por hoja (promedio)	240,0	338,1	265,1
Vaciados	-	1 953 618	753 963
Pares por lote (promedio)	-	158,0	8,4
Bajadas por par	-	8,83	2,95
Lecturas por par insertado	3,700	1,056	1,352
Escrituras por par insertado	1,012	1,059	1,355
ms por I/O de búsqueda	0,261	0,243	0,262

Tabla 4: Estructura de los árboles y contadores de la inserción con $N = 2^{26}$. *Bajadas por par*: total de pares bajados un nivel (sumando todos los lotes) dividido por N. La última fila es el tiempo de búsqueda dividido por sus I/Os.

3.4. Tamaño y altura

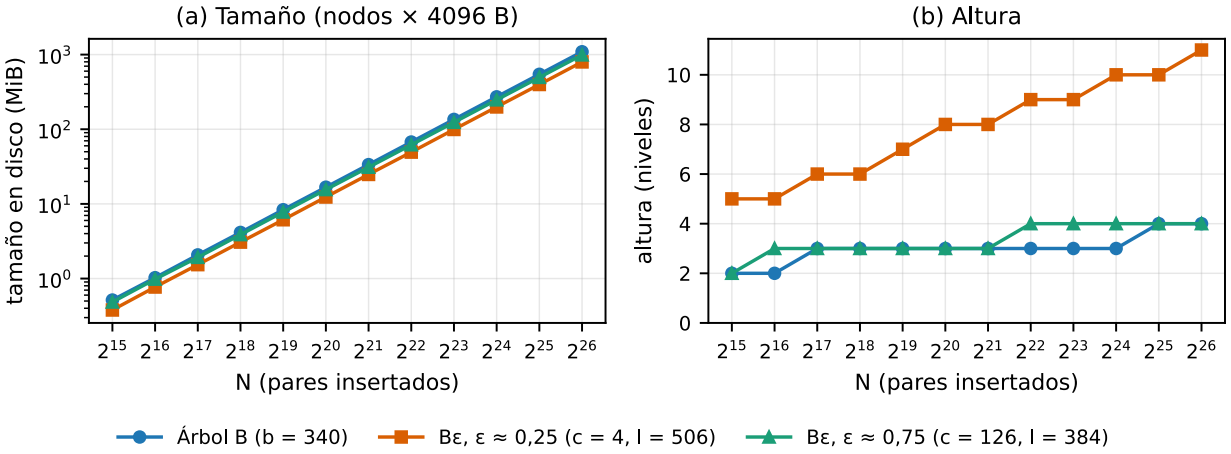


Figura 3: (a) Tamaño de los árboles (cantidad de nodos por 4096 bytes, que es el tamaño del archivo) y (b) altura, en función de N.

- El tamaño crece proporcionalmente a N (Figura 3 a). Con $N = 2^{26}$, el árbol B usa 279 471 nodos (1 091,7 MiB); el B^ε $c = 126$, 253 328 (989,6 MiB, 9 % menos), y el B^ε $c = 4$, 204 093 (797,2 MiB, 27 % menos). El orden es el mismo para todo N.
- La altura (Figura 3 b) del árbol B pasa de 2 a 4 (sube en 2^{17} y 2^{25}) y la del B^ε $c = 126$ de 2 a 4 (sube en 2^{16} y 2^{22}). La del B^ε $c = 4$ pasa de 5 a 11: sube un nivel cada una o dos duplicaciones de N.
- Las hojas del árbol B tienen en promedio 240 pares (71 % de 340); las del B^ε $c = 126$, 265 (69 % de 384), y las del B^ε $c = 4$, 338 (67 % de 506). Además, el 29,9 % de los pares del B^ε $c = 4$ está en buffers de nodos internos, contra 1,1 % en el de $c = 126$ y 0,4 % en los nodos internos del árbol B (Tabla 4).

3.5. Búsquedas

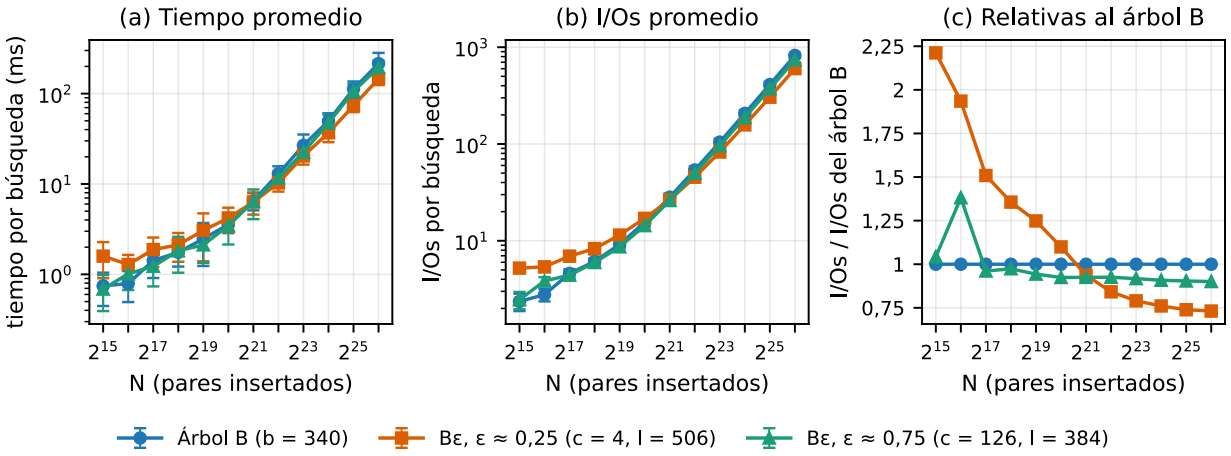


Figura 4: Búsquedas de rangos de una semana, promedio de 150 (50 rangos × 3 repeticiones), en función de N: (a) tiempo por búsqueda y (b) I/Os por búsqueda, con barras de ± una desviación estándar; (c) I/Os de cada árbol divididas por las del árbol B.

- Cada búsqueda devolvió en promedio el 0,291 % de los N pares, desde 97 pares con $N = 2^{15}$ hasta 195 420 con $N = 2^{26}$; los tres árboles devolvieron los mismos pares.
- Las I/Os por búsqueda (Figura 4 b) crecen casi proporcionalmente a N desde $N \approx 2^{19}$. Con N chico el B^ε $c = 4$ es el que más lee: 5,2 I/Os con $N = 2^{15}$, contra 2,4 del árbol B (2,2 veces) y 2,5 del B^ε $c = 126$. La relación se invierte hacia $N = 2^{21}$ (Figura 4 c): desde 2^{22}, el B^ε $c = 4$ es el que menos lee y con $N = 2^{26}$ hace 601,5 I/Os, contra 740,7 del B^ε $c = 126$ y 822,4 del árbol B (0,73 y 0,90 veces las del árbol B).
- El tiempo de búsqueda (Figura 4 a) sigue a las I/Os: con $N = 2^{26}$, 146 ms (B^ε $c = 4$), 194 ms (B^ε $c = 126$) y 215 ms (B), entre 0,24 y 0,26 ms por I/O (Tabla 4), que es la latencia medida para el disco. Hasta $N = 2^{20}$ el B^ε $c = 4$ es el más lento y el que más lee; desde 2^{21} es el más rápido y el árbol B el más lento.

3.6. Una misma búsqueda con dos valores de N

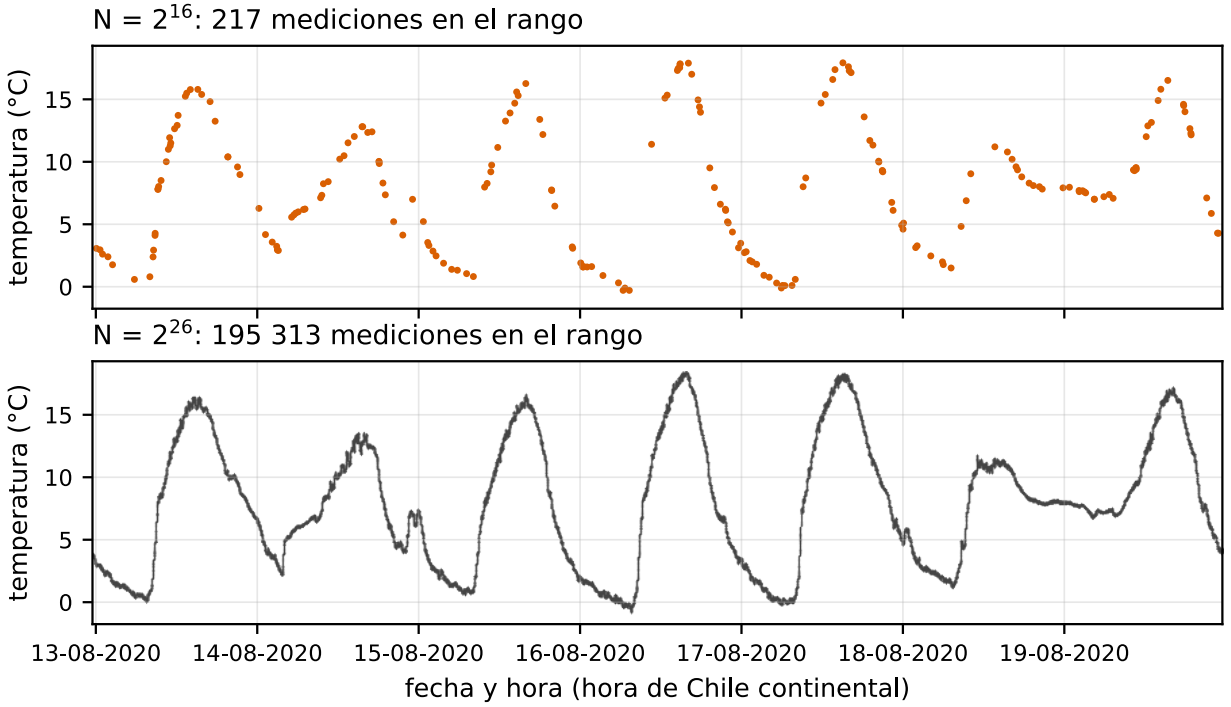


Figura 5: Pares que devuelve la búsqueda 0, [1 597 289 407, 1 597 894 207] (del 12-8-2020 a las 23:30 al 19-8-2020 a las 23:30, hora de Chile), con $N = 2^{16}$ y $N = 2^{26}$.

N	Pares	% de N	Mín. (°C)	Máx. (°C)	Prom. (°C)	Separación mediana (s)	¿Contenido en el de 2^{26}?
2^{16}	217	0,331	-0,29	17,92	8,21	2 214	sí
2^{20}	3 023	0,288	-0,77	18,32	8,00	140	sí
2^{26}	195 313	0,291	-0,80	18,33	7,86	2	sí

Tabla 5: Resumen de la búsqueda 0 para tres valores de N. *Separación mediana*: mediana del tiempo entre dos mediciones consecutivas del resultado.

La búsqueda 0 cubre una semana de invierno (Figura 5). Con $N = 2^{16}$ devuelve 217 mediciones, separadas por 37 minutos en la mediana, y con $N = 2^{26}$ devuelve 195 313, separadas por 2 segundos (Tabla 5). Las dos muestran el mismo ciclo diario: la temperatura sube durante la mañana hasta un máximo de 11,7 a 18,3 °C, en general a media tarde, y baja durante la noche, con mínimos cercanos a 0 °C en la madrugada en la mayoría de los días (el mínimo de la semana es −0,8 °C). La excepción es la noche del 18 al 19 de agosto, en que la temperatura se mantuvo entre 6,8 y 9,0 °C, y ambos valores de N lo muestran (con $N = 2^{16}$ hay 16 mediciones en ese tramo, entre 7,0 y 8,3 °C). El promedio de la semana cambia poco (8,2 y 7,9 °C). Todos los pares del resultado con $N = 2^{16}$ aparecen en el de $N = 2^{26}$. Con $N = 2^{16}$ se pierden las variaciones que duran menos de una hora, que con $N = 2^{26}$ se ven como una curva casi continua.

3.7. Bonus: elección de α

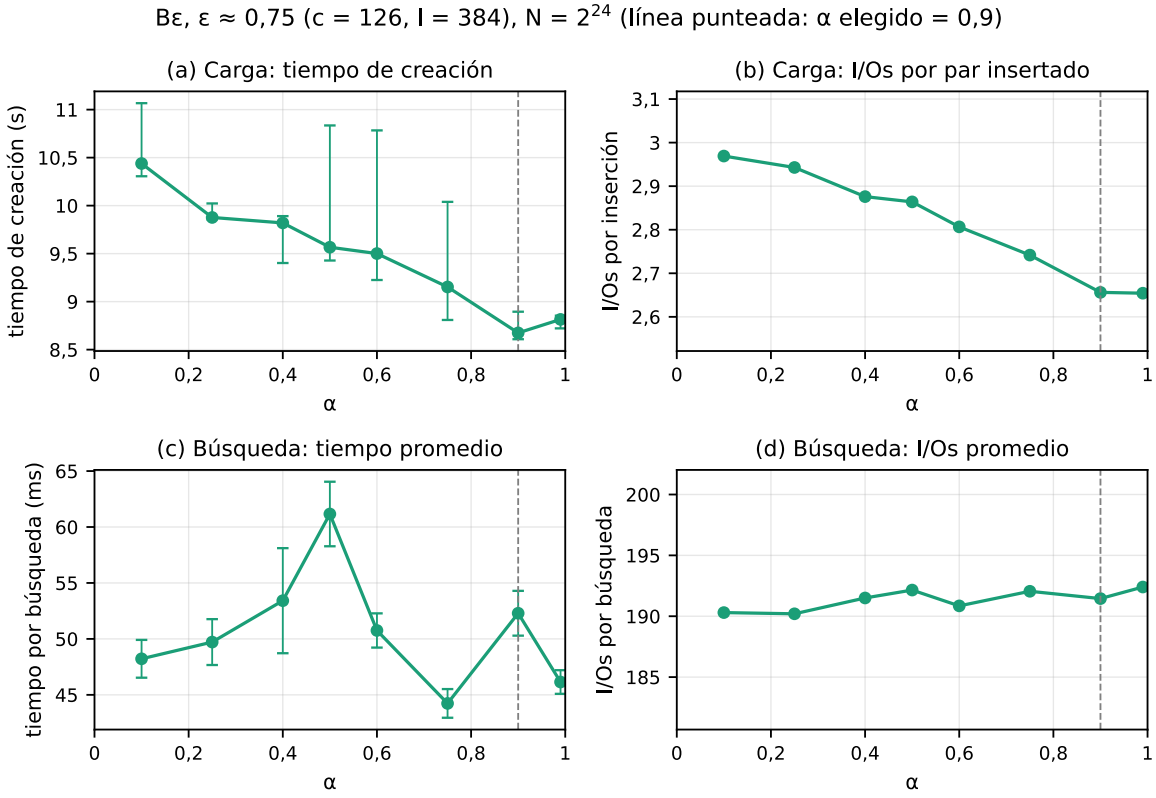


Figura 6: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$. (a) Tiempo de creación: mediana de 3 repeticiones, barras de mínimo a máximo; (b) I/Os por inserción; (c) tiempo por búsqueda: promedio de 60 (20 rangos × 3 repeticiones), barras de ± un error estándar; (d) I/Os por búsqueda. Las escalas de (b) y (d) van de 5 % bajo el mínimo a 5 % sobre el máximo.

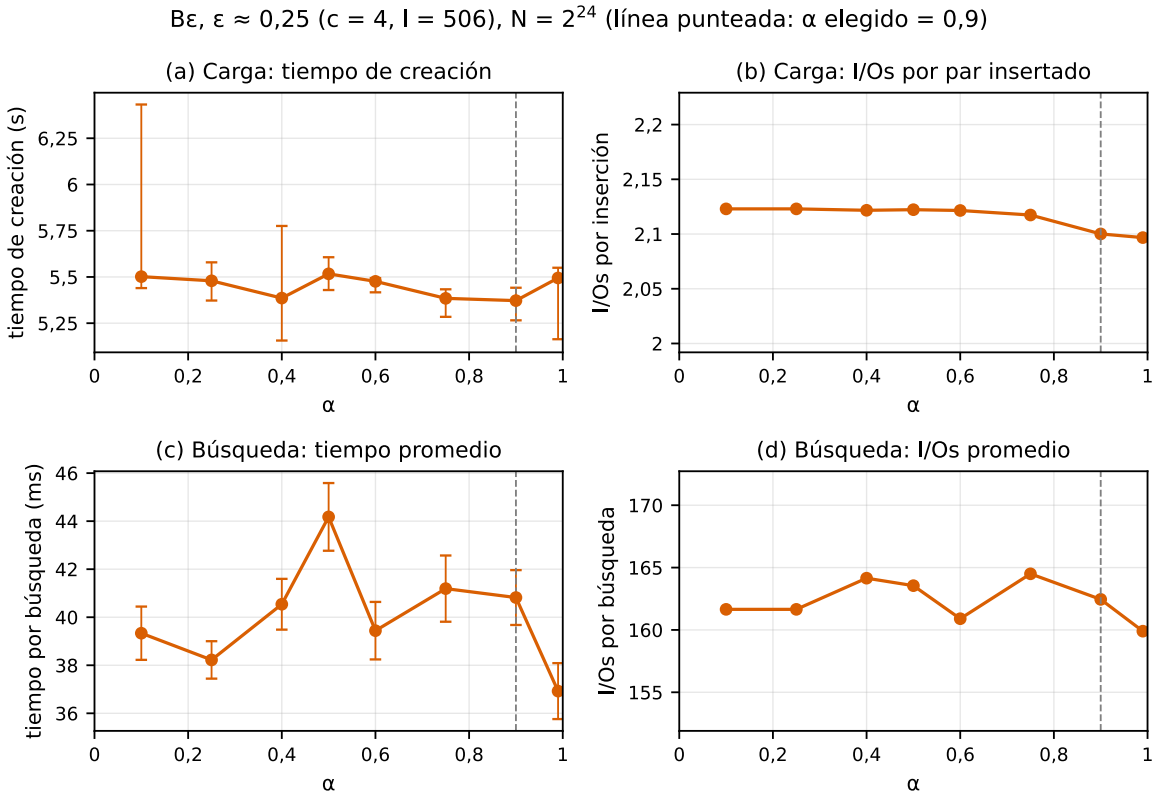


Figura 7: Experimento de α con el B^ε $c = 4$ y $N = 2^{24}$, con los mismos paneles que la Figura 6.

α	*t* creación (s)	desv. (s)	I/Os por par	Vaciados	Pares por lote	% pares en internos	Nodos	*t* búsq. (ms)	err. est. (ms)	I/Os búsq.
0,1	10,44	0,41	2,969	130 648	5,9	0,91	63 374	48,2	1,7	190,3
0,25	9,88	0,09	2,943	131 095	6,0	0,85	63 402	49,7	2,0	190,2
0,4	9,82	0,26	2,876	133 630	6,5	0,91	63 393	53,4	4,7	191,5
0,5	9,57	0,78	2,864	134 741	6,6	0,91	63 405	61,2	2,9	192,2
0,6	9,50	0,83	2,807	142 611	7,0	0,99	63 292	50,8	1,5	190,8
0,75	9,15	0,63	2,742	154 777	7,7	1,01	63 466	44,2	1,3	192,1
0,9	8,67	0,15	2,656	186 200	8,6	1,15	63 361	52,3	2,0	191,4
0,99	8,82	0,07	2,654	187 999	8,7	1,13	63 336	46,1	1,1	192,4

Tabla 6: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$. El tiempo de creación es la mediana de 3 repeticiones y *desv.* la desviación estándar de esas 3; el tiempo de búsqueda es el promedio de 60 búsquedas y *err. est.* su error estándar.

α	*t* creación (s)	desv. (s)	I/Os por par	Vaciados	Pares por lote	% pares en internos	Nodos	*t* búsq. (ms)	err. est. (ms)	I/Os búsq.
0,1	5,50	0,56	2,123	293 909	127,9	26,34	52 116	39,3	1,1	161,7
0,25	5,48	0,10	2,123	293 909	127,9	26,34	52 116	38,2	0,8	161,7
0,4	5,39	0,31	2,122	292 355	129,6	26,38	52 124	40,5	1,1	164,2
0,5	5,52	0,09	2,122	299 382	131,5	26,33	52 221	44,2	1,4	163,6
0,6	5,48	0,04	2,122	291 307	128,3	26,43	52 339	39,4	1,2	160,9
0,75	5,38	0,08	2,117	320 744	137,3	27,81	52 739	41,2	1,4	164,5
0,9	5,37	0,09	2,100	423 470	158,6	29,93	50 907	40,8	1,1	162,4
0,99	5,49	0,21	2,097	458 387	163,4	31,18	50 545	36,9	1,2	159,9

Tabla 7: Experimento de α con el B^ε $c = 4$ y $N = 2^{24}$, con las mismas columnas que la Tabla 6.

- **B^ε $c = 126$** (Figura 6, Tabla 6). Las I/Os de carga bajan de forma monótona con α: de 2,969 por inserción con $\alpha = 0{,}1$ a 2,656 con 0,9 y 2,654 con 0,99, es decir, 10,6 % menos (32 % menos sin contar la raíz). El lote promedio crece de 5,9 a 8,7 pares y los vaciados aumentan de 130 648 a 187 999. La mediana del tiempo de creación baja de 10,44 s a 8,67 s ($\alpha = 0{,}9$) y es 8,82 s con 0,99. Las I/Os de búsqueda quedan entre 190,2 y 192,4 (1,2 % de diferencia) y el tiempo de búsqueda entre 44,2 y 61,2 ms, sin tendencia.
- **B^ε $c = 4$** (Figura 7, Tabla 7). Las I/Os de carga bajan solo de 2,123 a 2,097 por inserción (1,2 %), y no de forma estrictamente monótona: entre 0,1 y 0,75 quedan entre 2,117 y 2,123 (con una subida

de 0,03 % de 0,4 a 0,5), y dos tercios de la baja ocurren entre 0,75 y 0,9. Con $\alpha = 0{,}1$ y 0,25 los archivos de los dos árboles son idénticos byte a byte. El lote promedio crece de 127,9 a 163,4 pares, los vaciados de 293 909 a 458 387 y la fracción de pares en nodos internos de 26,3 % a 31,2 %. El tiempo de creación queda entre 5,37 y 5,52 s, sin tendencia clara. Las I/Os de búsqueda varían entre 159,9 y 164,5 (2,9 %) y el tiempo entre 36,9 y 44,2 ms, también sin tendencia.

Elección. Se eligió $\alpha = 0{,}9$ para ambos árboles B^ε. Para la carga, el óptimo está en el extremo superior: desde $\alpha = 0{,}9$ las I/Os ya no bajan de forma apreciable (de 0,9 a 0,99, 0,07 % en $c = 126$ y 0,16 % en $c = 4$), y con $\alpha = 0{,}9$ se obtuvo la menor mediana del tiempo de creación en ambos árboles. Para la búsqueda, α no tiene un efecto medible: las I/Os cambian a lo más 1,2 % y 2,9 %, y las diferencias de tiempo son del tamaño del ruido. Por lo tanto, una mejor carga no empeoró la búsqueda. Entre 0,9 y 0,99, que en I/Os y tiempo quedan empatados, se prefirió 0,9 porque con $c = 4$ hace 8 % menos vaciados (423 470 contra 458 387) y por lo tanto menos trabajo de CPU. La Sección 4.3 explica estos resultados.

4. Análisis

4.1. Costo amortizado de la inserción

Árbol B. Cada inserción lee los h nodos del camino y escribe la hoja. Un split escribe 3 bloques y ocurre en promedio una vez cada ≈ 240 inserciones en una hoja (cada split crea una hoja nueva, y hay $N/240$ hojas), lo que suma $3/240 \approx 0{,}012$ escrituras por inserción: exactamente las 1,012 escrituras por par de la Tabla 4 (en las 36 construcciones del árbol B, las escrituras son $1 + N + 3 \cdot \text{splits}$). Por lo tanto, las I/Os por inserción son $h + 1{,}012$, lo que explica los escalones de la Figura 2 b: 3,00 con $h = 2$, $\approx 4{,}0$ con $h = 3$ y 4,71 con $N = 2^{26}$, donde el árbol llegó a $h = 4$ durante la construcción (cerca del par 20 millones, $N \approx 2^{24{,}3}$) y las primeras inserciones se hicieron con $h = 3$.

Árbol B^ε. Cada inserción cuesta 2 I/Os en la raíz. El resto son lotes: cada uno cuesta 2 I/Os (leer y escribir el hijo), más una escritura por nodo nuevo. En las 72 construcciones de B^ε se cumple exactamente que las lecturas son $N + \text{lotes}$ y las escrituras $N + \text{lotes} + \text{nodos}$, y los nodos son pocos (0,003 y 0,004 por inserción con $N = 2^{26}$). Si $\overline{m}$ es el tamaño promedio de un lote y $\overline{d}$ el número promedio de niveles que baja cada par,

$$\text{I/Os por inserción} \approx 2 + \frac{2\overline{d}}{\overline{m}}. \tag{1}$$

Con los contadores de la Tabla 4, la Ecuación 1 da $2 + 2 \cdot 8{,}83 \,/\, 158{,}0 = 2{,}112$ para $c = 4$ y $2 + 2 \cdot 2{,}946 \,/\, 8{,}377 = 2{,}703$ para $c = 126$, a 0,003 y 0,004 de lo medido (2,115 y 2,707). Esa diferencia son exactamente las escrituras de los nodos nuevos (0,003 y 0,004 por inserción), que la ecuación no incluye. La Figura 2 c muestra que ese término explica las I/Os sin la raíz para todo N.

El análisis amortizado acota ambos factores. Un par baja a lo más $h - 1 = O(\log_{B^\varepsilon} N) = O(\log_B N/\varepsilon)$ niveles. Y cada lote mueve muchos pares si el buffer está casi lleno al vaciarse, lo que se cumple siempre en la raíz (se vacía con $l_{\max}$ pares) y, como muestran los tamaños de lote de abajo, en la práctica también en los demás nodos (que se vacían cuando no les cabe un lote): con $\approx l_{\max}$ pares repartidos entre a lo más $c + 1 \approx B^\varepsilon$ hijos, el hijo más poblado recibe al menos $f \approx l_{\max}/(c+1) = \Omega(B^{1-\varepsilon})$ pares, y cada hijo elegido, al menos αf. Con un argumento de potencial, en el que cada par en un buffer guarda crédito para sus bajadas futuras y cada bajada cuesta $O(1/(\alpha B^{1-\varepsilon}))$ I/Os, el costo amortizado de los vaciados es

$$O\left(\frac{h}{\alpha B^{1-\varepsilon}}\right) = O\left(\frac{\log_B N}{\alpha\varepsilon B^{1-\varepsilon}}\right) \text{ I/Os por inserción.} \tag{2}$$

En los datos, con $\varepsilon \approx 0{,}25$ los lotes son grandes ($\overline{m} = 158$, cerca de $506/3{,}15 \approx 161$, porque los nodos internos tienen en promedio 3,15 hijos): aunque cada par baja 8,8 niveles, eso cuesta solo 0,11 I/Os. Con $\varepsilon \approx 0{,}75$ los lotes son de 8,4 pares: cada par baja solo 2,95 niveles, pero cada bajada cuesta $2/8{,}4 \approx 0{,}24$ I/Os, y el total es 0,70. La teoría predice que el costo del árbol B dividido por el de un B^ε (sin la raíz) es $\Theta(\varepsilon B^{1-\varepsilon})$: ≈ 27 para $\varepsilon = 0{,}25$ y $\approx 3{,}6$ para $\varepsilon = 0{,}75$. Lo medido con $N = 2^{26}$ es 32 y 5,3, del mismo orden. Como $\overline{d}$ crece con la altura, el costo sin la raíz de los B^ε crece lentamente con N (de 0,033 a 0,115 con $c = 4$, mientras la altura pasa de 5 a 11).

La raíz domina. En el modelo de la tarea la raíz se lee y se escribe en cada inserción, y esas 2 I/Os son el 95 % de las I/Os del B^ε $c = 4$ y el 74 % de las del B^ε $c = 126$. Por eso, contando todo, la ventaja de los B^ε sobre el árbol B es solo de 2,2 y 1,7 veces. En un sistema real la raíz (y probablemente los primeros niveles) estaría siempre en RAM, y la diferencia que importa es la de la Figura 2 c. Como ilustración, si cada I/O costara los $\approx 0{,}25$ ms por bloque medidos en este disco, insertar los 2^{26} pares directamente en disco tomaría unas 22 h con el árbol B (17 h con la raíz en RAM), mientras que con la raíz en RAM tomaría unas 3,3 h con el B^ε $c = 126$ y 32 minutos con el B^ε $c = 4$. Es una extrapolación gruesa, porque ignora la caché del sistema operativo y que las escrituras se pueden agrupar.

Tiempo. En la simulación, el tiempo de creación es aproximadamente las I/Os por el costo de copiar un bloque de 4 KiB: 0,10–0,16 µs por I/O con $N \leq 2^{20}$, cuando los árboles ($\lesssim$ 17 MiB) casi caben en la caché, y 0,18–0,26 µs con $N = 2^{26}$, cuando ocupan 0,8–1,1 GiB y lo más probable es que leer una hoja al azar falle en la caché y en la TLB (esto no se midió). El B^ε $c = 4$ es el más barato por I/O porque 2 de sus 2,1 I/Os son la raíz, que se usa en cada inserción y por eso se mantiene en la caché. Por eso la ventaja en tiempo es mayor que en I/Os: con $N = 2^{26}$, el árbol B tarda 3,2 veces lo que el B^ε $c = 4$ y hace 2,2 veces sus I/Os. El salto del árbol B en 2^{25} (Figura 1 b) coincide con su cuarto nivel.

4.2. Altura en función de ε y costo de búsqueda

Altura. En un árbol con H hojas cuyos nodos internos tienen en promedio F hijos, $h \approx 1 + \log_F H$. Con $N = 2^{26}$, F es 268 en el árbol B, 82 en el B^ε $c = 126$ (65 % de 127) y 3,15 en el B^ε $c = 4$ (cuyos nodos internos tienen entre 2 y 5 hijos). Eso da $\log_{268} 278\,430 = 2{,}2$, $\log_{82} 250\,234 = 2{,}8$ y $\log_{3{,}15} 139\,200 = 10{,}3$, del orden de los 3, 3 y 10 niveles internos medidos (alturas 4, 4 y 11). La fórmula es aproximada porque supone que todos los niveles tienen la misma cantidad de hijos, y cerca de la raíz no es así: por ejemplo, la raíz del árbol B tiene solo 4 hijos, y cada uno de ellos, entre 202 y 327 (leído del archivo del árbol). La razón entre los niveles internos de los dos B^ε (10 contra 3) es cercana a la razón 3 entre sus $1/\varepsilon$, como predice $h = O(\log_B N/\varepsilon)$. Como $F \approx 3$, el B^ε $c = 4$ debería sumar un nivel cada vez que N se multiplica por ≈ 3 (en los datos sumó 6 niveles entre 2^{15} y 2^{26}, uno por cada factor de $\approx 3{,}6$); los otros dos, cada vez que N se multiplica por 80 a 270.

Búsqueda. Un rango que cubre una fracción p del espacio de llaves intersecta $\approx p \cdot n_\ell + 1$ de los n_ℓ nodos de cada nivel ℓ, así que

$$\text{I/Os por búsqueda} \approx p \cdot n + h, \tag{3}$$

donde n es el total de nodos. Con $p = 604\,800/207\,705\,600 = 0{,}29$ %, la Ecuación 3 queda a menos de 2 % de lo medido (Tabla 8). Esto explica la Figura 4 c:

- Con N chico, $p \cdot n$ es menor que 1 y manda la altura: con $N = 2^{15}$, el B^ε $c = 4$ ($h = 5$) hace 2,2 veces las I/Os del árbol B ($h = 2$).
- Con N grande manda $p \cdot n$ (con $N = 2^{26}$, de 594 a 814 contra $h = 4$ a 11), y el orden lo da el tamaño. El B^ε $c = 4$ tiene 27 % menos nodos que el árbol B porque sus hojas guardan 338 pares en promedio contra 240, y porque guarda el 30 % de los pares en buffers de nodos internos, que en el árbol B solo sirven para dirigir la búsqueda. Las curvas se cruzan cerca de $N = 2^{21}$, cuando la diferencia de tamaño ($p \cdot n$: 25,0 contra 18,5) compensa la diferencia de altura (3 contra 8).

Por lo tanto, para estos rangos el árbol más bajo no es el más rápido, sino el más compacto. En una búsqueda puntual el costo sería solo la altura (4, 4 y 11 I/Os con $N = 2^{26}$), y el árbol B y el B^ε $c = 126$ serían casi 3 veces más baratos que el B^ε $c = 4$. El tiempo de búsqueda es casi exactamente las I/Os por la latencia del disco: entre 0,23 y 0,32 ms por I/O en todos los N y árboles, del orden de lo medido para el disco, así que revisar los buffers de los nodos visitados no tiene un costo de CPU apreciable.

N	Árbol	Nodos n	Altura h	$p \cdot n + h$	I/Os medidas	Diferencia (%)
2^{20}	B	4 287	3	15,5	15,5	0,2
2^{20}	B^ε c=4	3 164	8	17,2	17,1	-0,9
2^{20}	B^ε c=126	3 979	3	14,6	14,3	-1,7
2^{23}	B	34 675	3	104,0	104,6	0,6
2^{23}	B^ε c=4	25 425	9	83,0	82,6	-0,5
2^{23}	B^ε c=126	31 650	4	96,2	95,9	-0,2
2^{26}	B	279 471	4	817,8	822,4	0,6
2^{26}	B^ε c=4	204 093	11	605,3	601,5	-0,6
2^{26}	B^ε c=126	253 328	4	741,6	740,7	-0,1

Tabla 8: I/Os por búsqueda: modelo de la Ecuación 3 y valor medido (promedio de 150 búsquedas).

4.3. Efecto de α

Un lote cuesta 2 I/Os sin importar cuántos pares lleve, así que bajar grupos chicos es lo que desperdicia I/Os: si se dejan en el buffer, bajan más tarde junto con los pares que lleguen después. El umbral $\lceil \alpha f \rceil$ decide cuáles grupos bajan. Un α alto hace más vaciados, porque cada uno libera menos espacio, pero en esta implementación un vaciado no cuesta I/Os propias: la raíz se lee en cada inserción y un nodo interno se vacía cuando ya se leyó para recibir un lote. Solo cuesta CPU (agrupar el buffer). Por eso, en general, las I/Os de carga bajan al subir α: de forma monótona con $c = 126$, y salvo diferencias de 0,03 % con $c = 4$, donde α también cambia la forma del árbol.

El efecto depende de cuántos pares le tocan a cada hijo. Con $c = 4$, un vaciado reparte ≈ 500 pares entre unos 3 hijos, así que los grupos tienen alrededor de 160 pares. Con $\alpha \leq 0{,}25$ el árbol resultante es idéntico (los mismos contadores y el mismo archivo), lo que indica que esos umbrales eligen los mismos hijos; el lote promedio casi no cambia hasta $\alpha = 0{,}6$ (de 128 a 132 pares) y desde 0,75 crece hasta 163 pares, lo que ahorra poco, porque los lotes ya eran grandes. Con $c = 126$, en cambio, $f \approx 384/82 \approx 5$: con tan pocos pares por hijo, es esperable que el tamaño de los grupos varíe mucho de un hijo a otro. Con $\alpha = 0{,}1$ el umbral es $\lceil 0{,}5 \rceil = 1$ y baja todo grupo no vacío; con $\alpha = 0{,}9$ el umbral es 5 y los grupos chicos esperan. Ahí α cambia el costo en un 32 % sin contar la raíz.

En la búsqueda, α cambia cuántos pares quedan en buffers internos (de 26 % a 31 % con $c = 4$), pero esos nodos se leen igual, porque están en el camino de la búsqueda. Según la Ecuación 3 lo que importa es el número de nodos, que con α cambia a lo más 0,3 % con $c = 126$ y 4,3 % con $c = 4$, y las I/Os de búsqueda cambian a lo más 1,2 % y 2,9 %, respectivamente. Las diferencias de tiempo entre valores de α vienen de la latencia del disco: el tiempo por I/O de búsqueda varía entre 0,23 y 0,32 ms entre valores de α, mucho más de lo que varían las I/Os. Así, el compromiso que menciona el enunciado (mejor carga a cambio de peor búsqueda) no aparece en este experimento: la búsqueda no depende de α, y el criterio para elegirlo es la carga.

5. Conclusión

Se implementaron un árbol B y dos árboles B^ε con nodos de 4096 bytes. Se construyeron sobre un disco simulado que cuenta cada transferencia de bloque, se guardaron en un archivo y se buscaron leyendo de él con `O_DIRECT`, para $N = 2^{15}, ..., 2^{26}$ pares reales de temperatura. Todas las búsquedas se verificaron contra fuerza bruta y las invariantes de las estructuras se probaron por separado.

Hipótesis.

1. *Inserción: se cumplió.* El orden fue B^ε $c = 4$ < B^ε $c = 126$ < B en I/Os y en tiempo para todo N: con $N = 2^{26}$, 2,12, 2,71 y 4,71 I/Os por inserción, y 26, 44 y 83 s. Las I/Os de los B^ε están dominadas por las 2 de la raíz; sin ellas, la diferencia con el árbol B es de 32 y 5,3 veces, del orden de $\varepsilon B^{1-\varepsilon}$ que predice el análisis amortizado. La causa es el tamaño de los lotes: 158 pares por lote con $\varepsilon \approx 0{,}25$ contra 8,4 con 0,75.
2. *Altura y búsqueda: se cumplió en parte.* Las alturas fueron las esperadas (11 contra 4 y 4 con $N = 2^{26}$, con una razón de niveles internos cercana a la de $1/\varepsilon$), y con N chico el B^ε $c = 4$ fue el peor en búsquedas (2,2 veces las I/Os del árbol B con 2^{15}). Pero para $N \geq 2^{22}$ fue el mejor, no solo similar: con $N = 2^{26}$ hizo 0,73 veces las I/Os del árbol B. No se anticipó que una búsqueda que devuelve cientos de bloques cuesta $p \cdot n + h$, y que los buffers del B^ε también guardan datos, lo que lo hace el árbol más compacto.

3. *Tamaño: se cumplió.* El B^ε $c = 4$ usó 27 % menos bloques que el árbol B y el de $c = 126$ 9 % menos.

En el bonus, el costo de carga baja al subir α (de forma monótona con $c = 126$) y se estabiliza desde $\alpha \approx 0{,}9$, mientras que la búsqueda no depende de α. Por eso se usó $\alpha = 0{,}9$ en ambos árboles B^ε.

Limitaciones y mejoras.

- El modelo cuenta 2 I/Os de raíz por inserción, lo que esconde la mayor parte de la ventaja de los B^ε. Una versión futura podría mantener la raíz y los primeros niveles en un *buffer pool* (por ejemplo, LRU) y contar solo los bloques que salen de él, lo que se parecería más a un sistema real.
- Las inserciones se midieron en RAM. Medir algunas construcciones directamente en disco permitiría validar la extrapolación de la Sección 4.
- Solo se usaron rangos de una semana. Variar el ancho del rango (desde búsquedas puntuales hasta meses) mostraría en qué punto conviene cada árbol, según la Ecuación 3.
- Los tiempos se midieron en una máquina virtual compartida, con mediciones hasta 2 veces más lentas por otros procesos. Una máquina dedicada y más repeticiones reducirían ese ruido; las I/Os, en cambio, son exactas.
- No se implementó el borrado (no se pedía), ni la carga masiva de datos ordenados, que permitiría construir árboles completamente llenos.

Anexo: resultados completos del experimento principal

N	Árbol	t creación (s)	I/Os creación (millones)	I/Os por par	Nodos	MiB	Altura	t búsqueda (ms)	I/Os búsqueda	Resultados
2^{15}	B	0,010	0,10	3,002	132	0,5	2	0,75	2,4	97
	B^ε c=4	0,007	0,07	2,033	97	0,4	5	1,59	5,2	97
	B^ε c=126	0,009	0,07	2,221	124	0,5	2	0,69	2,5	97
2^{16}	B	0,020	0,20	3,007	263	1,0	2	0,79	2,8	191
	B^ε c=4	0,017	0,13	2,040	197	0,8	5	1,29	5,4	191
	B^ε c=126	0,019	0,15	2,303	251	1,0	3	1,01	3,8	191
2^{17}	B	0,049	0,44	3,389	531	2,1	3	1,42	4,6	380
	B^ε c=4	0,030	0,27	2,047	393	1,5	6	1,88	6,9	380
	B^ε c=126	0,041	0,31	2,351	492	1,9	3	1,23	4,4	380
2^{18}	B	0,113	0,97	3,700	1 062	4,1	3	1,75	6,1	761
	B^ε c=4	0,068	0,54	2,055	787	3,1	6	2,12	8,3	761
	B^ε c=126	0,087	0,62	2,368	992	3,9	3	1,82	6,0	761
2^{19}	B	0,295	2,02	3,856	2 139	8,4	3	2,47	9,2	1 525
	B^ε c=4	0,144	1,08	2,062	1 569	6,1	7	3,06	11,5	1 525
	B^ε c=126	0,193	1,25	2,385	1 991	7,8	3	2,12	8,7	1 525
2^{20}	B	0,656	4,13	3,934	4 287	16,7	3	3,57	15,5	3 049
	B^ε c=4	0,282	2,17	2,070	3 164	12,4	8	4,20	17,1	3 049
	B^ε c=126	0,404	2,54	2,424	3 979	15,5	3	3,46	14,3	3 049
2^{21}	B	1,499	8,33	3,973	8 601	33,6	3	6,52	28,2	6 086
	B^ε c=4	0,585	4,36	2,078	6 351	24,8	8	6,32	26,4	6 086
	B^ε c=126	0,883	5,24	2,497	7 930	31,0	3	6,39	26,1	6 086
2^{22}	B	3,176	16,75	3,993	17 266	67,4	3	12,98	53,9	12 207
	B^ε c=4	1,215	8,74	2,085	12 637	49,4	9	10,27	45,4	12 207
	B^ε c=126	1,899	10,84	2,586	15 843	61,9	4	11,67	49,9	12 207
2^{23}	B	6,315	33,58	4,003	34 675	135,4	3	26,76	104,6	24 448
	B^ε c=4	2,513	17,55	2,093	25 425	99,3	9	20,43	82,6	24 448
	B^ε c=126	4,194	22,13	2,638	31 650	123,6	4	22,48	95,9	24 448
2^{24}	B	13,317	67,24	4,008	69 581	271,8	3	49,86	207,1	48 865
	B^ε c=4	5,295	35,23	2,100	50 907	198,9	10	36,85	157,5	48 865
	B^ε c=126	8,896	44,56	2,656	63 361	247,5	4	47,27	188,0	48 865
2^{25}	B	35,119	148,03	4,412	139 394	544,5	4	112,72	411,8	97 706
	B^ε c=4	12,546	70,69	2,107	101 764	397,5	10	73,59	304,3	97 706
	B^ε c=126	23,001	89,68	2,673	126 696	494,9	4	106,19	372,2	97 706
2^{26}	B	83,197	316,22	4,712	279 471	1 091,7	4	214,97	822,4	195 420
	B^ε c=4	26,114	141,92	2,115	204 093	797,2	11	146,07	601,5	195 420
	B^ε c=126	44,337	181,67	2,707	253 328	989,6	4	194,33	740,7	195 420

Tabla 9: Resultados del experimento principal. El tiempo de creación es la mediana de 3 repeticiones; el tiempo, las I/Os y los resultados de búsqueda son promedios de 150 búsquedas. Las I/Os de creación son las del disco simulado y las de búsqueda son bloques leídos del archivo. Los B^ε usan $\alpha = 0{,}9$.